DOCUMENTAÇÃO TÉCNICA

Acompanhamento Diário

Organização e gestão do dia a dia do colaborador: demandas, apontamentos de horas, linha do tempo dos ciclos, rendimento e relatórios de prestação de contas.

VERSÃO
0.10

DATA
02/11/2026

REPOSITÓRIO
peubibytes/acompanhamento_diario

React 19 · TypeScript 7 · Vite 8 · Tailwind CSS 4 · TanStack Query 5 · React Router 8 · Node.js 26 · Express 5 · SQLite (node:sqlite) · Zod 4 · Vitest 5

Sumário

1 **Visão geral** — Propósito, módulos, conceitos e escopo

2 **Regras de negócio** — Período, meta, métricas, demandas, rendimento

3 **Arquitetura** — Monorepo, camadas e decisões

4 **Pacote `shared`** — Domínio compartilhado

5 **Back-end** — Banco, migrações, rotas e segurança

6 **Referência da API REST** — Endpoints, contratos e erros

7 **Front-end** — Telas, estado e sistema visual

8 **Execução e operação** — Instalação, variáveis, dados e backup

9 **Qualidade** — Testes, CI e convenções

10 **Histórico e roadmap** — Versões e próximos passos

Documento gerado a partir de `docs/fonte/documentacao.html`. A fonte da verdade das regras é o `docs/PLANO.md`.

1 Visão geral

O que a aplicação resolve, como está dividida e o que está dentro e fora do escopo.

1.1 Propósito

O **Acompanhamento Diário** nasceu para substituir uma planilha de controle de horas. Hoje é um organizador do dia a dia do colaborador: ele **declara** o que faz (chamados, reuniões, treinamentos e ciclos de trabalho longo), **aponta** o tempo gasto em cada demanda, acompanha a **linha do tempo** e os **impedimentos** dos ciclos e **presta contas** com o painel, o rendimento e o relatório para a daily ou o 1:1.

O apontamento de horas continua existindo, mas é **um módulo**, e não o núcleo. Toda funcionalidade nova precisa ajudar o colaborador a se organizar, priorizar ou prestar contas.

1.2 Módulos

MÓDULO	SITUAÇÃO	DESCRIÇÃO
Apontamento	Existente	Período sexta → quinta (ou mês), meta diária, apontamentos sobre demandas, métricas e conformidade.
Demandas	Existente	Tarefas (chamados, reuniões, treinamentos) e Ciclos (prazo, linha do tempo, impedimentos, bandeira).
Painel e rendimento	Existente	Dia útil, distribuição, tarefas, ciclos e rendimento, em semana ou mês.
Relatórios	Existente	Diário, semanal e mensal, com o indicador de trabalho não planejado.
Seus dados	Existente (v0.10)	Backup diário automático, exportar e importar.
Metas	Previsto	Objetivos com prazo e progresso.
Integração Azure	Futuro	Fora do escopo por enquanto.

1.3 Conceitos do domínio

TERMO	DEFINIÇÃO
Demanda	O que o colaborador declara que faz. Não tem horas. Duas famílias: Tarefas e Ciclos.
Tarefa	Demanda de rotina: *chamado* (número único, descrição, status), *reunião* (tipo, com quem, assunto; sem status) ou *treinamento* (nome, descrição, status).
Ciclo	Demanda de trabalho longo: tipo (planejada, intempestiva, preventiva), status, prioridade, início, prazo opcional, linha do tempo e impedimentos.
Apontamento	Intervalo de tempo (data, início, fim) gasto numa demanda declarada. No código e no banco, `activity`.
Período	Janela de tempo do apontamento e dos indicadores: a semana de **sexta a quinta** ou o **mês civil**. No código e nas rotas, `cycle` (a semana) e `period`.
Evento	Linha datada na linha do tempo de um ciclo: ação, decisão, impedimento, validação ou deploy.
Impedimento	Bloqueio de um ciclo, com responsável externo, início e resolução. A espera é contada em dias úteis.
Conformidade	Situação do dia em relação à meta: *Conforme* (exatamente a meta), *Não conforme* ou *Pendente* (sem apontamento).

Ciclo × Período. Até a v0.8 a interface chamava a semana de apontamento de "ciclo", o mesmo nome da demanda longa. Na v0.9 a semana passou a se chamar **Período** na interface; "Ciclo" ficou só para a demanda (decisão 0005).

1.4 Escopo

Incluído

- Declaração de demandas e apontamento sobre elas
- Métricas da planilha original: por dia, semana e mês
- Linha do tempo, impedimentos e bandeira dos ciclos
- Painel com cinco visões, em semana ou mês
- Relatórios do dia, da semana e do mês em Markdown, .md ou PDF
- Lembretes e agenda no sino
- Backup, exportação e importação dos dados
- Temas claro e escuro, celular e acessibilidade AA

Fora do escopo

- Integração com o Azure DevOps
- Login e múltiplos usuários: a aplicação roda localmente, para uma pessoa
- Lançamentos em sábados e domingos
- Folha de pagamento, cobrança a clientes, controle de ponto oficial

2 Regras de negócio

Todas as regras ficam no pacote `shared`, em TypeScript puro e testado; servidor e interface só as consomem.

2.1 Período

- **Semana:** começa na sexta-feira e termina na quinta seguinte; dias úteis Sex, Seg, Ter, Qua e Qui. O período de uma data começa na sexta mais recente ≤ data.
- **Mês:** o mês civil, com os dias úteis de segunda a sexta.
- Sábado e domingo não aceitam lançamentos. Datas trafegam como texto ISO (`AAAA-MM-DD`) e a aritmética é feita em UTC.

2.2 Meta diária

- 5h30, 9h ou outro valor em `hh:mm`. Meta semanal = meta diária × 5.
- Gravada por semana (`cycle_goals`): mudar a meta vale para a semana atual e as próximas; numa semana passada, só para ela.
- No mês, cada dia tem a meta da sua semana e a meta do período é a soma delas.

2.3 Métricas (réplica da planilha)

MÉTRICA	REGRA
Horas apontadas	União dos intervalos do dia: horários sobrepostos contam uma vez. Tempo sempre em minutos inteiros.
Diferença	Vazio se 0h · `00:00` se igual à meta · `+hh:mm` se excedeu · `-hh:mm` se faltou.
Conformidade	Pendente (0h) · Conforme (exatamente a meta) · Não conforme. Acima da meta é irregular e pede reajuste.
% preenchido	Horas do período ÷ meta do período, arredondado para baixo: 100% só com o período completo.
Dias conformes	`X/N`, com N = dias úteis do período (5 na semana).

2.4 Demandas

- **Declarar uma vez, apontar quantas vezes acontecer.** O apontamento exige uma demanda declarada e não tem status: o andamento é da demanda.
- Chamado repetido (mesmo número) ou reunião idêntica (tipo, participantes e assunto) → 409. O tipo de uma demanda não muda depois de declarado. Demanda com apontamentos não pode ser excluída.
- **Status das tarefas:** Não iniciado, Iniciado, Em andamento, Concluído. **Status dos ciclos:** Novo → Em andamento → Aguardando terceiro → Em homologação → Concluído | Cancelado; abertos = os quatro primeiros.
- **Histórico de status:** cada troca é gravada com a data; dele saem a data de conclusão e a última atualização (o mais recente entre evento, troca de status, impedimento e apontamento).
- **Prazo dos ciclos:** No prazo, Prazo próximo (até 2 dias úteis), Vence hoje, Atrasada; Sem prazo; Concluída ou Cancelada. Ciclo aberto sem atualização há 3 dias úteis aparece como *parado*.
- **Espera:** dias úteis com algum impedimento aberto, sem contar duas vezes os simultâneos; separada das horas apontadas (esforço).

2.5 Rendimento (v0.9)

- **Tarefas × Ciclos:** o tempo do período por família, por dia útil. Dentro de uma família, sobreposições contam uma vez; entre famílias, o trecho comum é dividido igualmente. Apontamentos antigos sem demanda formam o grupo "Sem demanda".
- **Por ciclo:** total no período, variação sobre o período anterior equivalente (minutos e %), registros da linha do tempo no período e, por dia, a média dos dias anteriores com apontamento.

2.6 Relatórios

Falam do **trabalho**, não dos apontamentos. O **diário** (v0.7) traz o que foi feito (ciclos, tarefas e reuniões), os impedimentos abertos ou resolvidos no dia e o que será feito no próximo dia útil; por padrão, o dia útil anterior. O **do período** (semana ou mês, v0.10) traz os ciclos que andaram com os eventos da linha do tempo, as tarefas concluídas e trabalhadas, as reuniões com quantas vezes aconteceram e os impedimentos do período. Montados na hora; nada é gravado.

2.7 Trabalho não planejado (v0.10)

Tempo em ciclos do tipo *Intempestiva* ÷ tempo apontado no período, com a mesma divisão de sobreposições do Tarefas × Ciclos. Só os ciclos têm tipo. Aparece no relatório do período e no Painel → Rendimento.

3 Arquitetura

Monorepo npm com três workspaces e uma regra simples: o domínio não depende de nada.

```
acompanhamento_diario/
├── shared/   regras de negócio em TypeScript puro (sem dependências), com testes
├── server/   Node + Express 5 + SQLite (node:sqlite); src/routes por domínio
├── web/      React 19 + Vite + Tailwind 4 + TanStack Query; testes de tela em web/test
├── scripts/  varredura anti-vazamento (check-sensitive)
└── docs/     PLANO.md (fonte da verdade), HISTORICO.md, decisions/ (ADRs) e esta documentação
```

3.1 Decisões técnicas

DECISÃO	MOTIVO
Tempo em minutos inteiros	Evita o arredondamento da planilha (`ROUND(x*1440)`) e erros de ponto flutuante.
SQLite via `node:sqlite`	Banco embutido no Node, sem dependência nativa para compilar; um arquivo, fácil de copiar.
TypeScript executado direto pelo Node	Sem etapa de build no servidor (type stripping).
Domínio em `shared`	Uma regra, um lugar: a interface mostra exatamente o que a API calcula, e os testes cobrem as regras sem HTTP nem tela.
Gráficos em SVG próprio	Controle total de acessibilidade, temas e tamanho do pacote.
Sem login, só loopback	Uso pessoal e local; expor na rede exige configuração explícita (seção 5.4).

3.2 Decisões registradas (ADRs)

Nº	DECISÃO
0001	Demandas ampliadas viram os chamados e os ciclos (v0.4).
0002	Demandas declaradas e apontamentos sobre elas (v0.5).
0003	Status só na demanda; histórico de status (v0.7).
0004	Impedimentos dentro da linha do tempo; bandeira (v0.8).
0005	"Período" no lugar de "ciclo de apontamento" na interface (v0.9).
0006	Fluxo de branches: uma entrega, um PR, uma tag por versão (v0.10).

4 Pacote `shared`

O domínio inteiro, importado como `@acomp/shared` pelo servidor e pela interface.

MÓDULO	RESPONSABILIDADE
`time.ts`	Conversão e formatação de tempo em minutos (`formatDuration` , `formatHM` , `parseHM`).
`cycle.ts`	Datas ISO, dias úteis e a semana sexta → quinta (`getCycle`).
`period.ts`	Semana ou mês: `getPeriod` , `shiftPeriod` , `previousPeriod` , `formatPeriodLabel` , `anchorDate` .
`metrics.ts`	Réplica das fórmulas da planilha: métricas do dia, da semana e do período, união de intervalos.
`demands.ts`	Categorias, status, prazo, espera, última atualização, ordenação e resumo das demandas.
`overview.ts`	Métricas de acompanhamento: tarefas concluídas, na fila e em andamento; ciclos abertos e parados.
`breakdown.ts`	Distribuição do tempo por demanda (rosca do Painel).
`yield.ts`	Rendimento: Tarefas × Ciclos, trabalho não planejado e rendimento por ciclo.
`timeline.ts`	Linha do tempo unificada (eventos e impedimentos).
`layout.ts`	Layout em serpentina do mapa da linha do tempo.
`reminders.ts`	Lembretes do sino e agenda.
`report.ts` · `periodReport.ts`	Relatório diário e relatório do período.
`types.ts`	Tipos comuns (apontamento, metas pré-definidas).

5 Back-end

Express 5 sobre SQLite, com validação em Zod e as regras do `shared`.

5.1 Banco e migrações

```
activities(id, date, start_minute, end_minute, project, description, notes, demand_id, …)
cycle_goals(cycle_start PRIMARY KEY, daily_goal_minutes)
settings(key PRIMARY KEY, value)
demands(id, category, title, description, ticket_number, meeting_type, participants,
        kind, priority, start_date, due_date, branch, pr_url, environment, status, …)
demand_events(id, demand_id, date, kind, text, flagged, created_at)
demand_blockers(id, demand_id, owner, description, since, resolved_at, flagged, created_at)
demand_status_changes(id, demand_id, from_status, to_status, changed_at, created_at)
```

O esquema evolui por **7 migrações** numeradas em `server/src/db.ts` (`PRAGMA user_version`), aplicadas ao iniciar o servidor. O banco usa WAL e chaves estrangeiras. Antes de migrar um banco existente, ele é copiado (`pre-migracao-N.db`).

5.2 Organização do servidor

ARQUIVO	PAPEL
`index.ts`	Lê as variáveis, abre o banco, agenda o backup diário e escuta em `HOST:PORT`.
`app.ts`	Monta a aplicação: segurança, limites de corpo, contexto comum e o registro dos grupos de rotas.
`http.ts`	Validações comuns (data, escala, id), resposta 400 e o contexto das rotas.
`routes/*.ts`	Um arquivo por domínio: períodos, relatórios, linha do tempo, apontamentos, demandas e dados.
`repo.ts`	Acesso ao banco (SQL), sem regra de negócio.
`backup.ts` · `transfer.ts`	Backups com `VACUUM INTO`; exportação e importação.
`security.ts`	Validação de `Host` e `Origin`, cabeçalhos e limites.

5.3 Backup, exportação e importação (v0.10)

- **Backup diário** ao iniciar e quando o dia vira: `backups/acompanhamento-AAAA-MM-DD.db`, gravado num temporário e renomeado; ficam os 14 mais recentes. Uma falha é registrada e não derruba o servidor.
- **Exportação:** JSON genérico (as colunas vêm do banco) com `app`, `schemaVersion` e `exportedAt`.
- **Importação:** só em banco vazio, ou com `?replace=true` depois de um backup `pre-importacao-…`. Exige a mesma versão do esquema, valida com Zod e roda numa transação.

5.4 Segurança (v0.10)

- Sem login: o servidor escuta só em `127.0.0.1`. Expor exige `HOST` e `ALLOWED_HOSTS`, com aviso no início.
- `Host` precisa ser da própria máquina ou liberado (barra o DNS rebinding); `Origin`, quando enviado, precisa ser igual ao `Host` (página de outro site → 403).
- JSON de até 100 kb (50 MB só na importação), sem `X-Powered-By`, com `nosniff` e `Referrer-Policy: no-referrer`.

6 Referência da API REST

JSON sobre HTTP em `/api`. Erros respondem `{ error }` com a primeira mensagem de validação em português.

MÉTODO	ROTA	USO
GET	`/cycles/:date`	Semana da data: 5 dias com métricas e totais.
PUT	`/cycles/:date/goal`	Meta diária da semana (e padrão das próximas).
GET	`/periods/:date?scale=`	Resumo do período (`week` ou `month`).
GET	`/metrics?date=&scale=`	Tarefas e ciclos abertos no período.
GET	`/yield?date=&scale=`	Rendimento: Tarefas × Ciclos, não planejado e por ciclo.
GET	`/activities?date=` ou `?from=&to=`	Apontamentos de um dia ou de uma faixa (até 62 dias).
POST PUT DEL	`/activities[/:id]`	Aponta, edita e remove; sobreposição gera aviso, não erro.
GET	`/demands?status=&category=`	Demandas com métricas e acompanhamento.
GET	`/demands/:id`	Demanda com apontamentos, eventos, impedimentos, histórico e linha do tempo.
POST PUT DEL	`/demands[/:id]`	Declara, edita e remove (repetida → 409; com apontamentos → 409).
POST PUT DEL	`/demands/:id/events[/:eventId]`	Linha do tempo (só ciclos).
POST PUT DEL	`/demands/:id/blockers[/:blockerId]`	Impedimentos (só ciclos).
PUT	`…/events/:id/flag`, `…/blockers/:id/flag`	Marca ou desmarca a bandeira.
GET	`/timeline?demandId=`	Linha do tempo unificada, do mais recente ao mais antigo.
GET	`/reports/daily?date=`	Relatório do dia (padrão: o dia útil anterior).
GET	`/reports/period?date=&scale=`	Relatório da semana ou do mês.
GET	`/export`	Todos os dados em JSON, como arquivo.
POST	`/import?replace=`	Importa uma exportação (400 inválida, 409 sem `replace` em banco com dados).
GET	`/backups`	Último backup diário, quantos há e quantos ficam.

Códigos. 400 entrada inválida ou JSON malformado · 403 `Host` / `Origin` não permitidos · 404 não encontrado · 409 conflito · 413 corpo grande demais · 500 erro interno.

7 Front-end

React com rotas, TanStack Query para o estado do servidor e um sistema visual próprio.

7.1 Telas

TELA	CONTEÚDO
Painel `/`	Escala Semana | Mês (na URL, `?escala=mes`), cinco visões (Dia útil, Distribuição, Tarefas, Ciclos, Rendimento), uma por arquivo em `pages/dashboard`, e navegação entre períodos.
Apontamentos `/apontamentos`	Abas por dia útil, lista por horário ou por demanda, formulário em painel deslizante.
Demandas `/demandas`	Tarefas | Ciclo, tipo e status; linhas compactas; linha do tempo e impedimentos nos ciclos.
Detalhes `/detalhes`	Dia a dia (tabela da planilha) e Ciclos (análise da linha do tempo, com filtros).
Relatório `/relatorio`	Escala Dia | Semana | Mês; relatório para copiar em Markdown, baixar ou imprimir.
Lembretes `/lembretes`	Só pelo sino: lembretes, agenda e "Seus dados" (último backup, exportar).

7.2 Estado e dados

- Data de referência e escala ficam na URL (`?data=`, `?escala=`), para sobreviver a recarregamentos e acompanhar a navegação entre telas.
- Preferências de apresentação (visão do Painel, lista por horário ou demanda) ficam no navegador.
- Toda leitura e escrita passa por `web/src/api/client.ts`, com invalidação das consultas afetadas após cada mutação.

7.3 Sistema visual

- Tokens em `web/src/index.css` (tipografia, raios, espaço, cores dos dois temas, paleta dos gráficos); nenhum valor solto.
- Um título, uma ação principal e um filtro por tela; textos explicativos no ⓘ (`InfoHint`), nunca fixos.
- Componentes comuns: `PageHeader`, `Segmented`, `SlideOver`, `InfoHint`, `States`, `PeriodNav`.
- Acessibilidade AA nos dois temas, foco visível, a cor nunca é a única pista, sem rolagem horizontal até 320 px.

8 Execução e operação

Roda localmente, sem serviços externos. O único requisito é o Node.js.

8.1 Requisitos e instalação

Node.js 26 (a versão do `.nvmrc`) e npm 11.

```
git clone https://github.com/peubibytes/acompanhamento_diario.git
cd acompanhamento_diario
npm install

npm run dev      # desenvolvimento: API em 127.0.0.1:3001 e interface em http://localhost:5173
npm start        # uso diário: build da interface + servidor único em http://127.0.0.1:3001
```

8.2 Scripts

SCRIPT	AÇÃO
`npm run dev`	Servidor com `node --watch` e Vite com HMR, em paralelo.
`npm test`	Regras, API e varredura (`test:rules`, `node --test`) e telas (`test:web`, Vitest).
`npm run typecheck`	`tsc` em `shared`, `server`, `web` e nos testes de tela.
`npm run check:sensitive`	Varredura anti-vazamento (planilhas, bancos, chamados reais, e-mails).
`npm run build` · `npm start`	Build da interface; build e servidor numa só porta.
`npm run docs:pdf`	Regenera este PDF (requer Edge ou Chrome).

8.3 Variáveis de ambiente

VARIÁVEL	PADRÃO	USO
`PORT`	`3001`	Porta do servidor.
`HOST`	`127.0.0.1`	Interface em que o servidor escuta.
`ALLOWED_HOSTS`	vazio	Nomes de acesso liberados além da máquina, separados por vírgula.
`DB_PATH`	`server/data/acompanhamento.db`	Arquivo do banco.
`BACKUP_DIR`	`server/data/backups`	Pasta dos backups.

8.4 Restaurar um backup

1. Pare o servidor.
2. Apague `acompanhamento.db-wal` e `acompanhamento.db-shm`, se existirem.
3. Copie o backup escolhido por cima de `server/data/acompanhamento.db`.
4. Inicie o servidor; migrações pendentes rodam sozinhas, com cópia antes.

Privacidade. O repositório é público. Banco, backups e a planilha de referência nunca são versionados, e testes e exemplos usam só dados fictícios. A varredura `check:sensitive` roda no CI.

9 Qualidade

Testes das regras, da API e das telas, tipagem estrita e CI em todo PR.

9.1 Testes

SUÍTE	COBERTURA
`shared/test`	Tempo, semana e mês (viradas de mês e ano, bissexto), métricas da planilha, demandas, prazo, espera, acompanhamento, distribuição, rendimento, linha do tempo, serpentina, lembretes e relatório.
`server/test`	API real sobre banco em memória: CRUD, validações, conflitos, períodos e rendimento; segurança (`Host`, `Origin`, limites); backup e rotação; exportação e importação.
`scripts`	A varredura anti-vazamento acusa o que deve e não expõe valores.
`web/test`	Telas em jsdom contra a API real: navegação entre períodos, escala e visões do Painel, escala do Relatório, formulário de apontamento e declaração de chamado.

Total: **221 testes** (204 de regras, API e varredura; 17 de tela).

9.2 Integração contínua

GitHub Actions (`.github/workflows/ci.yml`) em todo PR e push na `main`: `npm ci`, varredura, typecheck, testes e build, com o Node do `.nvmrc`. Dependências com versão exata; atualização mensal, uma dependência maior por vez.

9.3 Convenções

- TypeScript em modo `strict`; imports com extensão `.ts` no servidor e no domínio.
- Identificadores em inglês; interface, mensagens, comentários e commits em português.
- Regras só em `shared`; o servidor valida e persiste; a interface apresenta.
- Uma branch por entrega, PR para a `main`, tag por versão (ADR 0006). Mudança de comportamento atualiza o `PLANO.md` no mesmo PR.

10 Histórico e roadmap

Versões entregues e próximos passos. O detalhe de cada versão está no `PLANO.md` e no `HISTORICO.md`.

VERSÃO	ENTREGA
0.1	Apontamento de horas: semana sexta → quinta, meta, métricas da planilha, painel e detalhes.
0.2	Demandas de ciclo, sobreposição contada uma vez, distribuição do dia.
0.3	Painel sob demanda, sino e tela de lembretes e agenda.
0.4	Demandas ampliadas: status, eventos, impedimentos e espera.
0.5	Demandas declaradas (Tarefas e Ciclos) e apontamentos sobre elas.
0.6	Reestilização: tokens, ⓘ, painel deslizante, acessibilidade AA.
0.7	Status só na demanda, histórico de status, métricas de tarefas e relatório diário.
0.8	Linha do tempo unificada, bandeira, análise em Detalhes e mapa em serpentina.
0.9	"Período", escala Semana | Mês e visão Rendimento.
0.10	Segurança de acesso, backup e exportação, README, CI, testes de tela, organização do código, relatório do período e trabalho não planejado.

Próximos passos

- Metas com prazo e progresso; importador do texto legado; validações novas no sino.
- Integração com o Azure DevOps (futuro).